# 时钟之迹

从时钟源，到一次延时

STM32F407ZGT6

Keil MDK / STM32CubeMX

168 MHz

1 ms

HAL 默认滴答 · 时间从这里开始

01 初始化

02 设置

03 获取

04 应用

代码追踪：CH5EX01HAL / CH5Ex01STD

追踪路线

# 四个阶段，一条时间线

01

## 初始化

程序从哪里开始

`SystemInit()`
`HAL_Init()`

02

## 设置

时钟与滴答多快

`SystemClock_Config()`
`SysTick_Config()`

03

## 获取

当前频率与时间

`HAL_RCC_Get…Freq()`
`HAL_GetTick()`

04

## 应用

什么时候做动作

`Delay()`
`HAL_Delay()`

**CH5EX01HAL** 外部时钟配置 8 MHz

**CH5Ex01STD** 外部时钟配置 25 MHz

两份实际例程，主频均配置为 168 MHz；先追函数，再核对参数。

源码：HAL main.c、hal_conf.h；STD Usermain.c、system_stm32f4xx.c、stm32f4xx.h

# 初始化：先找到真正的入口

复位 → `SystemInit()` → C 运行时初始化 → `main()`　　两份工程共同的入口

## 标准库 • main 之前

`SystemInit()`

→ `SetSysClock()`

→ `HSE + PLL + 总线分频`

进入 main 时，主时钟配置已经完成。

## HAL • main 中

main.c:73 / 80 / 87

```
73 HAL_Init();
80 SystemClock_Config();
87 MX_GPIO_Init();
```

先建默认时基，再设置 168 MHz。

---

切换时钟后：HAL_RCC_ClockConfig() 更新 SystemCoreClock，并再次调用 HAL_InitTick()。

源码：两工程启动文件与 system_stm32f4xx.c；HAL main.c:73–87、hal_rcc.c:718–722

# 设置：8 MHz 怎样变成 168 MHz

| HSE | | M | | N | | P | | SYSCLK |
|---|---|---|---|---|---|---|---|---|
| 8 | ÷ | 8 | × | 336 | ÷ | 2 | = | **168** MHz |

CH5EX01HAL / main.c:136–140

```
136 RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
137 RCC_OscInitStruct.PLL.PLLM = 8;
138 RCC_OscInitStruct.PLL.PLLN = 336;
139 RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
140 RCC_OscInitStruct.PLL.PLLQ = 4;
```

## 同一个 VCO，另一条支路

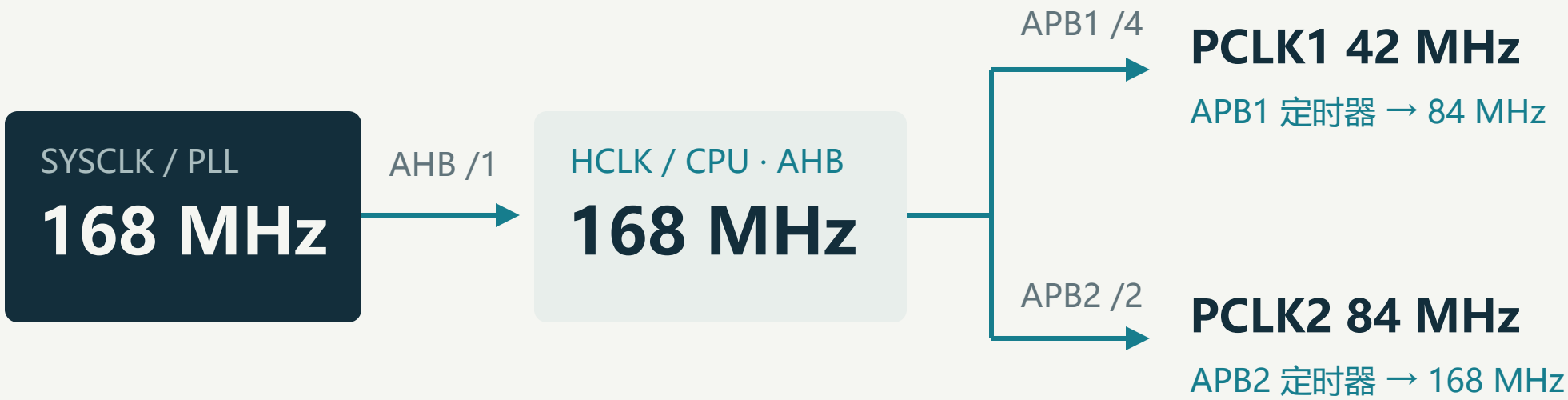
原代码：336 ÷ 4 = 84 MHz

USB 48 MHz → Q 应设为 7

标准库：25 ÷ 25 × 336 ÷ 2
同样得到 168 MHz；Q 原值为 7。

源码：HAL main.c:136–140；STD system_stm32f4xx.c:371、384、401–403；RM0090 §6

# 设置：主频沿总线继续分配




```
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
```

原例程随后以 FLASH_LATENCY_5 调用 HAL_RCC_ClockConfig()。

## F407 的 TIM 时钟规则

APB 分频 = 1：TIM = PCLK

APB 分频 ≠ 1：TIM = 2 × PCLK

源码：HAL main.c:150–155；原理：RM0090，STM32F405/407 RCC 时钟树

# 获取：频率与时间，是两个量

## 读时钟频率 Hz

| | |
|---|---|
| `HAL_RCC_GetSysClockFreq()` | 168 000 000 |
| `HAL_RCC_GetHCLKFreq()` | 168 000 000 |
| `HAL_RCC_GetPCLK1Freq()` | 42 000 000 |
| `HAL_RCC_GetPCLK2Freq()` | 84 000 000 |

标准库在 Usermain.c:48 读取整组频率

```
RCC_GetClocksFreq(&RCC_Clocks);
```

## 读累计时间 ms

`HAL_GetTick()`

返回时基计数 uwTick。
默认每次中断累计 1 ms。

stm32f4xx_hal.c:323–326

```
__weak uint32_t HAL_GetTick(void)
{
  return uwTick;
}
```

SystemCoreClock 记录 HCLK；SystemCoreClockUpdate() 根据寄存器更新这个变量，硬件时钟配置保持不变。

源码：HAL hal_rcc.c 的 Get…Freq()；hal.c:312–326；STD Usermain.c:48

# 设置 SysTick：一拍究竟多长

## 24 位递减计数器：装载 → 递减 → 到零中断 → 重装

`T = (LOAD + 1) / fSysTick`

两份例程均通过 CMSIS 选择 HCLK

### 标准库 · HCLK / 100

**10 ms**

ticks = 1 680 000　LOAD = 1 679 999

### HAL 默认 · HCLK / 1000

**1 ms**

ticks = 168 000　LOAD = 167 999

```
SysTick_Config(RCC_Clocks.HCLK_Frequency / 100);
HAL_SYSTICK_Config(SystemCoreClock / (1000U / uwTickFreq))
```

原调用表达式；HAL 默认 uwTickFreq = 1。CMSIS 写入 LOAD = ticks − 1；改 HCLK 后需重新配置滴答。

源码：STD Usermain.c:49；HAL hal.c:256；CMSIS core_cm4.h:2022–2036；PM0214 SysTick

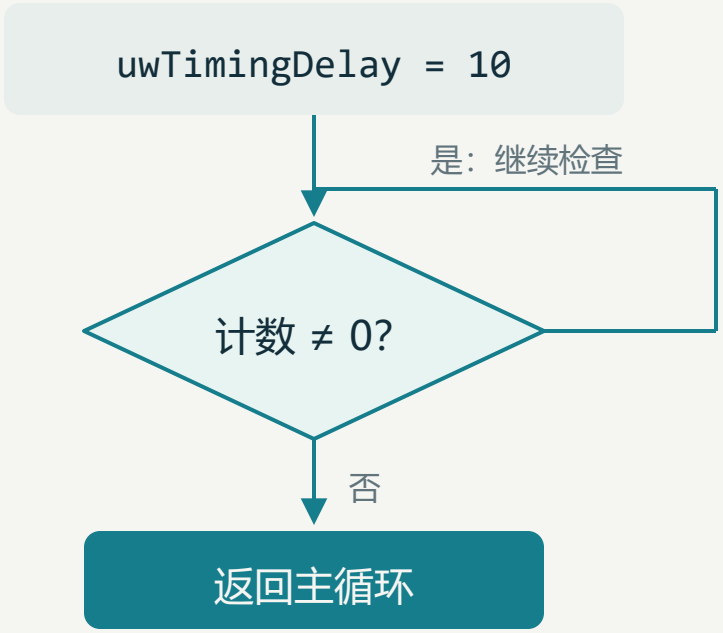
# Delay：等一个倒计数归零

Usermain.c:83–100

```
void Delay(__IO uint32_t nTime)
{
  uwTimingDelay = nTime;
  while(uwTimingDelay != 0);
}

void TimingDelay_Decrement(void)
{
  if (uwTimingDelay != 0x00)
  {
    uwTimingDelay--;
  }
}
```

Delay(10) · 主循环




每 10 ms：SysTick_Handler() → TimingDelay_Decrement() → 非零计数减 1

Delay(10) ≈ 100 ms · Delay(1) ≈ 10 ms · 参数表示当前时基的滴答数

源码：STD Usermain.c:20、49–53、83–100；stm32f4xx_it.c:142–145

# HAL_Delay：等累计时间走够

stm32f4xx_hal.c:389–403

```
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();
  uint32_t wait = Delay;
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }
  while((HAL_GetTick() - tickstart) < wait)
  {
  }
}
```

默认多等待一个滴答，保证实际等待不少于请求时长。

HAL_Delay(1000)

**1** 秒左右

参数单位：毫秒

① HAL_InitTick：配置时基

② HAL_IncTick：累计时间

③ HAL_GetTick：读取毫秒

④ HAL_Delay：比较经过时间

SysTick_Handler → HAL_IncTick
经过时间 ≥ wait 时返回。

源码：HAL main.c:100；hal.c:312–326、389–403；stm32f4xx_it.c:184–190

# 应用：一段延时，怎样变成闪烁

## 标准库 • 每 5 次循环翻转

```
Delay(10);
if(Index%5==0)
  GPIO_ToggleBits(GPIOE, LED2_Pin);
```

Usermain.c:62–65；Index 在循环末尾递增。

翻转间隔 ≈ **500 ms**

流水灯约每 100 ms 移到下一位。
稳定运行后，完整亮灭周期约 1 s。

## HAL • 每次循环翻转

```
HAL_Delay (1000);

HAL_GPIO_TogglePin(GPIOE, LED2_Pin);
```

main.c:100–102

翻转间隔 ≈ **1 s**

流水灯约每 1 s 移到下一位。
完整亮灭周期约 2 s。

一次翻转只完成半个亮灭周期。精确定时用 TIM / PWM；主循环兼顾多项任务时，用 tick 差值安排动作。

源码：STD Usermain.c:60–74；HAL main.c:94–110。

# 把时钟放回各自的任务里

## 高速源与 PLL

HSI：内部 16 MHz RC，可用于启动与后备。
HSE：外部晶振，提供主时钟参考。
PLL：对参考时钟分频、倍频，再分配输出。

## 系统与总线时钟

SYSCLK：选定的系统时钟源输出。
HCLK：CPU、AHB 总线的节拍。
PCLK1 / PCLK2：APB 外设的节拍。

## 低速时钟

LSE：外部 32.768 kHz，适合 RTC 走时。
LSI：内部约 32 kHz，供独立看门狗等使用。

## 软件计时与硬件定时

SysTick：软件时基、超时判断、短时等待。
TIM / PWM：周期中断、精确定时与波形输出。

```
__HAL_RCC_GPIOE_CLK_ENABLE();
```

使用外设前，先打开对应的时钟门控。

gpio.c:48 · 未使用的外设可关闭时钟，以降低功耗。

依据：RM0090，STM32F405/407 RCC、RTC、TIM；PM0214 SysTick；HAL gpio.c

# 从初始化到应用，线索已经连起来

看懂一个 Delay，需要把整条时钟链连起来。

### 初始化

先追入口

SystemInit / HAL_Init

### 设置

再算频率与周期

PLL / 总线 / SysTick

### 获取

随后读配置与计数

Get...Freq / GetTick

### 应用

最后看动作

Delay / HAL_Delay

标准库一拍 **10 ms** · HAL 一拍 **1 ms** · 调用之前，先确认单位

代码：教师例程 CH5EX01HAL、CH5Ex01STD（主程序、RCC/HAL驱动、中断与CMSIS文件）。

手册：ST RM0090 · STM32F405/407 RCC 章节；ST PM0214 · Cortex-M4 SysTick 章节。